11. Python: Tuples

"Lock it in, speed it up."

Previous Section:

</> 10. Python: Lists

Contents:

Tuples are another important data structure in Python. At first glance, they may look very similar to lists because both can store multiple items in a single variable. However, there is one major difference:

- Lists are designed to be dynamic and changeable.
- Tuples are designed to be fixed and protected.

In other words, once we create a tuple, we normally do not change its contents. This makes tuples useful when we want data to remain constant throughout the program.

1. What is a Tuple?

A tuple is a collection that stores multiple items in a single variable.

Unlike lists, tuples are **immutable**, which means we cannot insert elements,, remove elements, or modify existing elements directly.

For example: `(1, 2, 3, 4, 5)`. A tuple is usually enclosed using parentheses `()`.

Although tuples are immutable, we can still perform many built-in operations on them such as:

```
len(tuple)
sum(tuple)
min(tuple)
max(tuple)
```

We can create tuples using either `()` or the `tuple()` constructor.

Example:

```
tuple1 = ('Apple', 'Microsoft', 'Google', 'Yahoo')

print("Length:", len(tuple1))

# Indexing
print('Element at position 0:', tuple1[0])
print('Element at position 1:3:', tuple1[1:3])
```

```
Length: 4
Element at position 0: Apple
Element at position 1:3: ('Microsoft', 'Google')
```

Since tuples cannot be modified directly, if we want to append or change elements, we usually convert the tuple into a list first.

```python
tuple1 = ('Apple', 'Microsoft', 'Google', 'Yahoo')

# Convert tuple to list
tuple1 = list(tuple1)
# Append element
tuple1.append('Facebook')

# Convert back to tuple
tuple1 = tuple(tuple1)

print(tuple1)
```

('Apple', 'Microsoft', 'Google', 'Yahoo', 'Facebook')

2. Methods on Tuples

Most concepts such as indexing, slicing, looping, membership checking (`in`), and nested structures work very similarly to lists.

```
tuple1 = ('Apple', 'Microsoft', 'Google', 'Yahoo')

# Indexing
print(tuple1[0])
# Slicing
print(tuple1[1:3])

# Looping
for company in tuple1:
    print(company)

# Membership checking
print('Google' in tuple1)
```

```
Apple
('Microsoft', 'Google')

Apple
Microsoft
Google
Yahoo

True
```

- We can also create nested tuples:

```
student = (('Iris', 20), ('Luna', 21), ('Aether', 19))

print(student[0])
print(student[1][0])
```

```
('Iris', 20)
Luna
```

However, because tuples are immutable, many list methods such as: `append()` , `remove()` , `pop()` , `sort()`

do not work on tuples.

Example:

```
tuple1 = (5, 2, 8, 1)

# These methods do not work on tuple
tuple1.append(10)
tuple1.remove(2)
tuple1.pop()
tuple1.sort()

print(tuple1)
```

AttributeError: 'tuple' object has no attribute 'append'

If we try to use these methods directly, Python will produce an error because tuples cannot be modified. Instead, we usually convert the tuple into a list first:

```
tuple1 = (5, 2, 8, 1)

# Convert tuple to list
temp = list(tuple1)

# Modify the list
temp.append(10)
temp.sort()

# Convert back to tuple
tuple1 = tuple(temp)

print(tuple1)
```

(1, 2, 5, 8, 10)

So overall, tuples behave similarly to lists, but with fewer modification-related methods because their contents are designed to stay fixed.

Summary

Tuples are immutable collections used to store multiple values safely and consistently. While they look similar to lists, tuples focus more on stability rather than flexibility.

In general:

- Use lists when the data needs to change frequently.
- Use tuples when the data should remain fixed and protected.

Understanding tuples is important because many Python functions and systems return data in tuple form, especially when working with multiple return values, databases, and data processing.

References

https://www.geeksforgeeks.org/python/python-tuples/

https://www.programiz.com/python-programming/tuple

Next Section:

</> 12. Python: Sets